Lab Programming 14

Alpha-Beta Pruning Algorithm for Gaming

Aim

To write a Python program to implement the Alpha-Beta Pruning algorithm for a simple game.

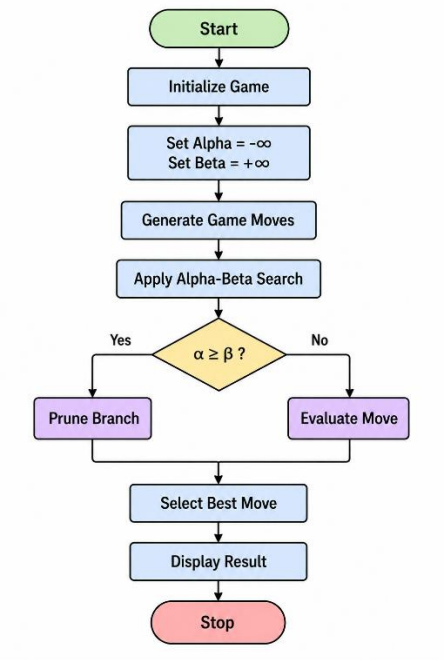
Objective

To implement Alpha-Beta pruning.

Algorithm

1. Start.
2. Create the game states.
3. Set alpha = -∞ and beta = +∞.
4. Evaluate possible moves using Minimax.
5. Prune branches when beta <= alpha.
6. Select the best move.
7. Display the result.
8. Stop.

Flowchart

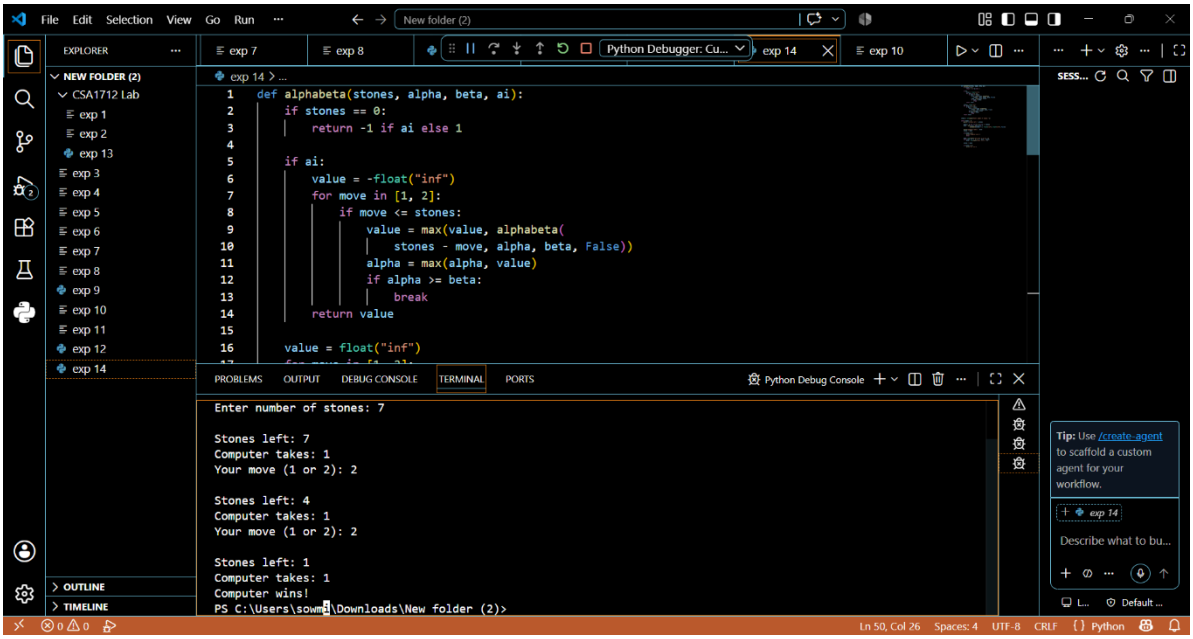


Code

```
def alphabeta(stones, alpha, beta, ai):
    if stones == 0:
        return -1 if ai else 1
    if ai:
        value = -float("inf")
        for move in [1, 2]:
            if move <= stones:
                value = max(value, alphabeta(
                    stones - move, alpha, beta, False))
                alpha = max(alpha, value)
                if alpha >= beta:
                    break
        return value
```

```
    value = float("inf")
    for move in [1, 2]:
        if move <= stones:
            value = min(value, alphabeta(
                stones - move, alpha, beta, True))
            beta = min(beta, value)
            if alpha >= beta:
                break
    return value
stones = int(input("Enter number of stones: "))
while stones > 0:
    print("\nStones left:", stones)
    moves = [m for m in [1, 2] if m <= stones]
    move = max(moves, key=lambda m:
              alphabeta(stones - m, -float("inf"), float("inf"), False))
    print("Computer takes:", move)
    stones -= move
    if stones == 0:
        print("Computer wins!")
        break
    move = int(input("Your move (1 or 2): "))
    while move not in [1, 2] or move > stones:
        move = int(input("Enter 1 or 2: "))
    stones -= move
    if stones == 0:
        print("You win!")
```

Output



Result

The game was successfully implemented using Alpha-Beta pruning.

Conclusion

Alpha-Beta pruning reduces unnecessary searches in the Minimax algorithm and helps the computer choose the best move efficiently.